Deterministic introduction for iPRA

Eliminating the supplied witness, with an explicit structural bootstrap

Research prepared for Vladimir Reshetnikov

November 11, 2026 (Pacific time)

Abstract

The current ConciseIPRA proposals already reduce arithmetic and full intuitionistic logic to unusually small proof interfaces. We prove a further restriction of the guarded presentation: a paired introduction need not carry a witness term, parameter index, or witness-choice bit. Its witness is read from a fixed literal naming matrix when that pattern occurs; otherwise it is the newest numerical parameter, with zero as the empty-scope default.

The restriction preserves every open consequence, not merely the closed theorems. The main issue is that numerical-context weakening can change the automatically selected witness. We resolve it by proving algebraic cut first, then repairing weakening by an induction on the original derivation, and only afterwards deriving general cut and unrestricted introduction. Composition with the pinned guarded/native and native/iPRA bridges gives whole-domain bi-interpretability, actual equality, identity numerical composite graphs $x = y$, and equiconsistency with full first-order iPRA.

This is a reduction in independent inference data, not a claim of a globally smallest axiomatization. Complete executable checkers and their measured costs are included. Separate, small implementation refinements of the inherited compact checkers are identified as such; their sizes are not attributed to the new mathematical calculus. No new Lean or Rocq verification is claimed.

Contents

1 Result, comparison, and the meaning of "simpler" 2

2 The complete presentation and its equivalence proof 3

3 Why the result is full iPRA bi-interpretability 7

4 A failed shortcut and the boundary it exposes 9

5 Further research questions 10

6 Executable implementation and honest size comparison 11

7 Reproduction and deliverable map 13

8 Complete checker for the new calculus 14

1 Result, comparison, and the meaning of "simpler"

The comparison is pinned to ProveIt commit `11814771b7490dfbea0ed5fcf0f5b23586334764`, under `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. It includes the seventy-three filed packages described by that snapshot's README and the six archives then present in `docs/incoming`. References [1, 7] identify the exact sources. The delivered provenance file records their identities and hashes.

The mathematical change is narrow and explicit. A previous introduction contains an arbitrary term t supplied by the proof author. Our introduction contains no such datum. It obtains t by a deterministic operation on the matrix and the ordered numerical context. Both versions have the same two computation schemas and the same four logical/equality inference families. We do not count a restriction on an existing rule as deletion of another rule.

Three different comparisons must be kept distinct:

1. **Inference interface.** The new rule deletes the independent witness field, including the special choice of a parameter index.

2. **Matched complete checker.** Recognizing the naming pattern is part of the trusted implementation. Its cost is measured against an otherwise matched unrestricted-witness checker below; it is not free.

3. **Repository source-size frontier.** The incoming archives already contain a 508-byte compact checker and a 780-byte version with its native public-goal validator. The coordinate minima 256 tokens, 276 AST nodes, and 508 bytes belong to different programs with different contracts. They are attained sizes, not proved lower bounds.

Accordingly, the mathematical result should be judged as a proved restriction of witness choice. A numerical saving obtained by changing Python diagnostics or integer expressions is a separate implementation result. Neither proves that no simpler iPRA presentation exists.

What is inherited. The scalar iterator, paired binder D, guarded formula grammar, full formula transport, clause contexts, ordinary logical reconstruction, and arithmetic compiler all come from the pinned repository. Existential naming through equations is also standard and appears in earlier proposals. The new theorem is that a literal naming exception and a deterministic newest-parameter default suffice for *all* introductions, including the structural admissibility argument required by that default.

Verification boundary. The universal equivalence result below is a mathematical proof. The code is checked by executable certificates, adversarial examples, and comparison with the matched source calculus. Those checks test implementations; they are not a proof-assistant verification of the metatheorem. The repository's existing downstream Lean bridge is cited precisely, and is not presented as a formalization of this new rule.

2 The complete presentation and its equivalence proof

The new calculus $\mathcal{G}_{\mathrm{d}}$ is defined here without relying on an informal notion of a proof step. Its numerical scopes are finite ordered lists of distinct parameters. Retained clauses are a separate finite list; adding or deleting a clause does not alter the parameter list. The displayed schemas range over finite, well-scoped syntax, with capture-avoiding substitution and fresh bound names. There is no computation normalization in syntax, no external theorem oracle, and no implicit equality decision.

2.1 The inherited guarded calculus

Terms are the raw terms

$$t ::= a \mid 0 \mid S(t) \mid I(n, r; u.h),$$

where only u is bound in h. No computation normalization is built into substitution. Put $C(v, r, s) = I(v, r; u.s)$ with u fresh for s, and put $H = (I(0, 0; u.0) = 0)$. A canonical formula is either an equation or $D_{x,y}B$, where B is compound or, if B is an equation, has the literal shape

$$I(y, 0; u.h) = I(y, 0; v.k). \tag{G}$$

The bound variables x, y are distinct. The state variables u, v are bound only in their respective update terms. Updates may depend on x, y and all outer parameters. Capture-avoiding substitution for outer parameters preserves this grammar. The designated contradiction is $0 = S0$.

A retained clause $\langle A; y.B(y)\rangle$ denotes $A \to \forall y\, B(y)$; y is absent from A. An ordinary assumption A is the constant clause $\langle H; v.A\rangle$ with unused v. Every context admits the two computation axiom schemes

$$I(0, r; u.h) = r, \qquad I(Sn, r; u.h) = h[I(n, r; u.h)/u].$$

Besides these, there are four inference families:

Use. For a retained $\langle A; y.B(y)\rangle$, infer

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}.$$

The numerical target t is any term in the current numerical scope.

Transport. For a canonical formula template $F(v)$, infer

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}.$$

Open. With b fresh for the outer context and conclusion, infer

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, \langle B(b, 0); y.B(b, y)\rangle \vdash A}{\Gamma \vdash A}.$$

Unrestricted introduction (comparison calculus only). Infer

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ an equation}, \\ 0, & B \text{ compound}. \end{cases}$$

In Intro, z is fresh for Γ, t and all outer matrix parameters. This is precisely the raw canonical guarded-core presentation of the pinned repository, with full formula transport; it is not the later term-only transport kernel. Its ordinary reading is

$$D_{x,y}B \;:=\; \exists x(B(x,0) \to \forall y\, B(x,y)).$$

The previous guarded-core report supplies its translations and both composites with the native calculus and hence with full first-order iPRA. Those results are inherited here, rather than reasserted as a new proof from scratch.

2.2 A witness determined by syntax and scope

For an outer-scoped term r define the literal naming matrix

$$N_r(x,y) := C(y,0,x) = C(y,0,r). \qquad \text{(Name matrix)}$$

Both updates are state-independent. In particular r contains neither matrix variable x, y nor either freshly introduced state variable. Every such matrix satisfies (G). Recognition is literal modulo bound-variable renaming; it is not equality up to provability or computation.

An explicit numerical scope. Keep the ambient finite list of distinct numerical parameters separate from the retained clause context. Write it as $\vec{a} = (a_1, \dots, a_n)$. Open appends its fresh witness to this list in its branch. Introduction appends its step eigenparameter in its branch. Additional clauses do not by themselves change the numerical scope. Define

$$\omega(B,\vec{a}) = \begin{cases} r, & B \text{ is literally } N_r \text{ with } r \text{ outer scoped,} \\ a_n, & B \text{ is not such a matrix and } n > 0, \\ 0, & B \text{ is not such a matrix and } n = 0. \end{cases} \qquad \text{(DW)}$$

The newest parameter is read from the *parent* scope, before the fresh step eigenparameter is allocated. A successful naming match takes precedence. The theory $\mathcal{G}_{\mathrm{d}}$ has the same syntax, axioms, Use, Tr and Open, and has only this introduction:

$$\frac{\Gamma, B(\omega(B,\vec{a}), \eta_B(z)) \vdash B(\omega(B,\vec{a}), Sz)}{\Gamma \vdash D_{x,y}B}. \qquad \text{(DI-d)}$$

Its certificate has a matrix and one branch proof, with no witness field.

Complete inventory. The new presentation consists of the stated term/formula grammar, the two computation axiom schemas, Use, full formula Tr, Open, and DI-d. It has no additional primitive hypothesis, reflexivity, weakening, cut, coverage, successor, ordinary connective, or induction rule. Ordinary assumptions are retrieved by Use with the fixed computation theorem H. The unrestricted introduction displayed earlier defines only the comparison calculus $\mathcal{G}$ and is not an extra rule of $\mathcal{G}_{\mathrm{d}}$.

2.3 The structural bootstrap

The structural problem. Changing the ambient numerical scope can change the selected witness. Thus ordinary weakening is not justified by a blind rule-by-rule replay. In particular, one must not use general cut, whose proof needs weakening of the cut donor, to prove weakening of this calculus. The following bootstrap uses a restricted cut first.

Lemma 2.1 (Algebraic proofs and restricted cut)**.** *Call a proof algebraic if it uses only the computation axioms, Use and Tr, with no introduction or Open. Such proofs admit injective parameter renaming and arbitrary extensions or reindexings of their available clauses and parameters. Moreover, an ordinary assumption in an arbitrary $\mathcal{G}_{\mathrm{d}}$ proof can be cut against an algebraic donor proof.*

Proof. For the first assertion, induct on the algebraic proof. Its rules have no scope-dependent choice and no newly bound eigenparameter. For the second, induct on the proof receiving the cut. A Use of the deleted constant clause is replaced by the algebraic donor, extended and renamed by the first assertion. At DI-d and Open, freshen the local variable and continue the induction in the extended branch. The numerical parameter list is unchanged when the ordinary-assumption clause is deleted, so DI-d selects exactly the same witness before and after the deletion. This proves restricted cut without general weakening of proofs containing DI-d. □

Lemma 2.2 (Reflexivity and symmetry)**.** *For every term r, $r = r$ is derivable using only the zero computation and Tr; from $s = t$ the equation $t = s$ is derivable with the same resources.*

Proof. Put $q = I(0, r; u.0)$. The computation equation $q = r$, used as both premises of Tr with template $F(v) = (v = r)$, gives $r = r$. From $s = t$ transport $s = s$ with template $F(v) = (v = s)$ to obtain $t = s$. □

Lemma 2.3 (Every term has a name)**.** *For every well-scoped term r, $\mathcal{G}_{\mathrm{d}}$ proves $D_{x,y}N_r(x, y)$. Opening this theorem at a fresh parameter b supplies a context in which $b = r$ is derivable without any use of an unrestricted witness.*

Proof. The deterministic naming alternative reads r from the matrix. It asks for

$$\Gamma, N_r(r, z) \vdash N_r(r, Sz).$$

The conclusion is literally $C(Sz, 0, r) = C(Sz, 0, r)$, so reflexivity proves it independently of the local induction assumption. This is a permitted progressive introduction on a guarded equation; no numerical induction is needed in its proof.

Open the resulting theorem. Its clause is $\langle N_r(b, 0); y.N_r(b, y)\rangle$. Both terms in $N_r(b, 0)$ compute to 0, so the antecedent follows from the two zero computations and Tr. Use at $1 = S0$ gives $C(1, 0, b) = C(1, 0, r)$. The successor computation for a state-independent update gives $C(1, 0, b) = b$ and $C(1, 0, r) = r$. Symmetry and Tr therefore give $b = r$. All these computations are literal instances of the two stated axioms. The extracted equation and its symmetric form have algebraic proofs from the opened clause. □

Theorem 2.4 (Weakening and injective renaming for determined introduction)**.** *Let a finite $\mathcal{G}_{\mathrm{d}}$ derivation be given. Embed its parameters injectively into any larger ordered parameter list, and embed its clauses, after that renaming, into any larger clause context. There is an effectively constructed derivation of the correspondingly renamed conclusion in the larger context and scope.*

Proof. Induct on the height of the *original* derivation, with the parameter and clause embeddings universally quantified. Shared computation, Use and Tr cases are immediate. At Open, choose a receiving fresh witness, extend the embeddings, and apply the induction hypotheses to its proper premises.

Consider a DI-d node with matrix B and original selected witness w. Let B' and w' be their images under the given injective parameter map. Literal naming matches are preserved and reflected

by such maps: bound slots cannot become free parameters or conversely, and a free parameter cannot become a fixed numeral or a term constructor. If $B = N_r$, the new selected witness is exactly the renamed $r = w'$, and the branch induction hypothesis gives the rule directly. If the default selected witness is also unchanged, the same direct construction works.

Suppose, therefore, that B' is not a naming matrix and its new default witness differs from w'. Prove $DN_{w'}$ by the naming lemma and open it at a receiving fresh parameter e, which is now newest. In this branch its name clause gives the algebraic proof $e = w'$. We will introduce DB' at e; this is exactly the witness DI-d now selects. Let v be its fresh step parameter, and write

$$A_{\mathrm{old}} = B'(w', \eta_{B'}(v)), \qquad A_{\mathrm{new}} = B'(e, \eta_{B'}(v)).$$

Apply the weakening induction hypothesis to the original DI-d node's *proper branch*, mapping its old step variable to v. Use a receiving context containing the desired outer clauses, the name clause, and both A_{old} and A_{new} as ordinary assumptions. This supplies $B'(w', Sv)$ there. This use of the induction hypothesis is legitimate even though e and v are newly allocated receiving parameters: the induction statement quantifies over all embeddings of the smaller original proof.

From A_{new}, transport along the algebraically proved $e = w'$ to obtain A_{old}. This is an algebraic donor. Restricted cut removes the extra A_{old} assumption, giving $B'(w', Sv)$ with only the intended local assumption A_{new}. Algebraic symmetry and one further Tr give $B'(e, Sv)$. Apply DI-d at newest e and discharge its name by Open.

Every recursive call was on a proper subderivation of the original proof. The newly built naming and transport proofs were never inputs to that recursive weakening transformation. Restricted cut was already established. Thus the construction is well founded and has no circular appeal to general cut or weakening. The same argument includes a changed closed-scope default $w = 0$, in which case the added name is N_0. □

Corollary 2.5 (Ordinary-assumption cut)**.** *General cut of an ordinary assumption is admissible in* $\mathcal{G}_{\mathrm{d}}$.

Proof. Induct on the proof receiving the cut, replacing uses of the deleted constant clause by the supplied donor. The theorem now supplies the donor's weakening and fresh renaming at every insertion site. Deleting only a clause leaves the numerical parameter list unchanged, so existing determined introductions keep their selections. □

Theorem 2.6 (The fully determined introduction is complete)**.** *For every finite canonical clause context* Γ *and canonical formula* A*, in each ambient numerical scope,*

$$\Gamma \vdash_{\mathcal{G}_{\mathrm{d}}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{G}} A.$$

Both transformations are effective and preserve the entire end sequent.

Proof. The forward direction regards the selected witness as the witness of the old Intro rule. For the converse, induct on the original $\mathcal{G}$ proof. At an old introduction with a literal naming matrix N_r, ignore its supplied witness and branch and use the independent naming theorem DN_r. At any other old introduction, translate its proper branch by induction, prove and open DN_t for its old witness t, and let b be this new fresh parameter. In the context containing the opened name and the new ordinary assumption $B(b, \eta_B(z))$, transport along $b = t$ produces $B(t, \eta_B(z))$. General weakening and ordinary-assumption cut insert the translated old branch, giving $B(t, Sz)$. Transport along $t = b$ gives $B(b, Sz)$. The newest parent parameter is b, and B is not a naming matrix, so

DI-d now selects exactly b. Close that introduction and the name opening. The structural theorem and general cut were proved before this comparison, so the simulation has no circularity. □

Corollary 2.7 (Full bi-interpretability and equiconsistency)**.** *The ordinary realization of $\mathcal{G}_{\mathrm{d}}$ is bi-interpretable and equiconsistent with full first-order iPRA. Both numerical domains are whole, equality is actual equality, and both numerical composite maps have graph $x = y$.*

Proof. The new comparison is the identity on all terms, formulas, clauses and ordinary readings. Compose it with the pinned guarded/native and native/iPRA interpretations described in Section 3. Their existing formula round-trip derivations transfer without changing their statements. Neither numerical domain nor numerical map changes. The contradiction $0 = S0$ is preserved, giving equiconsistency. □

Arbitrary numerical substitution is admissible through the exact comparison; unchanged certificate replay after substitution is not claimed.

2.4 Why the qualifications matter

Why the root-selected guarded basis is useful. The simulation changes the chosen witness from an arbitrary term t to a parameter b. It is essential that this does not change the assumption of the branch. The older rule that chooses progress exactly when the witness is literal zero cannot simply be substituted into this argument: at $t = 0$ it would use $B(0, z)$, whereas at b it would use $B(b, 0)$. The guarded-core mode is independent of the witness. The substitution-stable diagonal-guard basis also has this property and supports the same naming construction.

What the newest-parameter proof adds. The stronger theorem does remove the index field. Its extra mathematical cost is exactly the preceding structural proof: ordinary numerical weakening may require elaboration through a new name and algebraic transports. The executable fixtures include a proof valid before scope extension, its failed naive replay in a larger scope, and a repaired proof obtained by naming the old selected parameter anew.

No implicit equality decision. Testing $B = N_r$ is a fixed syntactic match, including the absence of the new bound names from r. Replacing that by "some row names the witness" would need extra row data or proof search; a universal rule about a provable naming row would reintroduce an equality-proof premise. The narrow pattern is sufficient for completeness and is the simpler mathematical specification.

3 Why the result is full iPRA bi-interpretability

It would be insufficient to show that the new rules hold over the standard natural numbers. It would also be insufficient to recover the equations of primitive recursive functions while ignoring quantified intuitionistic formulas. Here the new comparison preserves the entire open consequence relation, and the remaining interpretation chain is the one already established in the pinned sources.

3.1 The guarded/native comparison

Let $\mathcal{C}$ be the unrestricted Choice-DI calculus: it has raw scalar terms, Eq/D formulas, the two computation schemas, Use, full formula Tr, Open, case introduction at every matrix, and progress introduction at literal equations. Its ordinary reading is the one stated for D above.

For an equation $E = (s = t)$ define

$$G_v E = (I(v, 0; u.s) = I(v, 0; u.t)), \qquad L(E) = D_{a,v} G_v E,$$

where the added variables are fresh. Define $P(A) = L(A)$ when A is an equation and $P(A) = A$ otherwise. Then $P(A)$ is always compound. The guarded-core proof [2] establishes, under arbitrary retained contexts,

$$L(E) \dashv\vdash E.$$

Its forward direction opens the clause, supplies the true zero row, and specializes at 1. Its reverse direction uses the retained E to prove the successor row, ignoring the induction hypothesis.

The structural translation into the guarded language is

$$T(s = t) = (s = t), \qquad T(D_{x,y} B) = D_{x,y} P(TB).$$

Literal inclusion is the interpretation in the other direction. Case introductions translate through padding. Equation progress translates using the guarded calculus's derived equation-induction rule. An opened padded clause is used by padding its antecedent and unpadding its consequence. These are transformations of arbitrary finite open derivations.

The two formula composites are also proved, not assumed. In the unrestricted calculus, padding and general binder congruence give $A \dashv\vdash JT(A)$. In the guarded calculus, compound matrices use compound binder congruence; guarded atomic matrices use their provable zero row to establish

$$D_{x,y} B \dashv\vdash D_{x,y} L(B).$$

This is why the zero guard in the public grammar is substantive. Dropping it would invalidate this argument. The source proof, supplied in `reference/guarded_core.tex`, gives the equation-induction bootstrap and both composites in detail. The new deterministic calculus proves exactly the same guarded sequents, so every one of those derivations transfers literally.

3.2 The ordinary scalar theory

The intermediary T_{eq} uses ordinary intuitionistic first-order logic with equality over scalar terms. Its nonlogical principles are

$$I(0, a; u.h) = a, \quad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$

$$\forall x\, (x = 0 \vee \exists y\, x = Sy), \qquad 0 = S0 \to \bot,$$

together with contextual induction for literal equations. "Contextual" means the retained side assumptions may themselves be quantified formulas; only the induction formula is restricted to an equation.

The ordinary realization sends $D_{x,y} B$ to $\exists x (B(x, 0) \to \forall y\, B(x, y))$ and sends a clause to $A \to \forall y B$. In T_{eq}, compound case introduction is justified by coverage; guarded atomic progress is justified by equation induction and the true zero row. Equality transport and clause application use the usual intuitionistic equality and quantifier rules. Thus the reading is sound.

Conversely, the native logical macros recover implication, conjunction, disjunction, both quantifiers, and falsity, with their introduction and elimination rules, and prove the formula round trips. This is the full native/ordinary bridge [3, 4, 6]. No excluded-middle rule for arbitrary formulas is used. The code of a primitive recursive description is never an object-language numerical variable ranging over all programs.

3.3 The arithmetic interpretation and its inverses

In iPRA, each fixed scalar term has a primitive recursive interpretation. An occurrence $I(n, a; u.h)$ becomes the primitive recursion that starts at the translation of a and repeatedly applies the already translated update h, ignoring the recursion counter in that update. This is external recursion over a fixed finite term description, not an internal universal evaluator.

In the reverse direction the repository develops, in order, a scalar predecessor, reflection of actual equality into a Boolean-valued term, quantifier-free induction, pairing with projections, and a compiler for arbitrary fixed primitive-recursive descriptions [5]. For a recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

the scalar compiler stores the counter and current output in one coded state. Writing π for pairing and ℓ, r for its two projections, put

$$W(u, \vec{p}) = \pi(S\ell(u), h^*(\ell(u), r(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = r(V(n, \vec{p})).$$

The invariant $\ell(V(n, \vec{p})) = n$, proved by equation induction, and the projection equations prove the two defining equations internally. Initial functions and composition translate directly.

Equality reflection compiles each quantifier-free formula A to a fixed term b_A with $b_A = 0 \leftrightarrow A$ and a proved Boolean range. Equation induction on $b_A = 0$ therefore yields full quantifier-free induction, including under arbitrary side assumptions. This is an inherited bootstrap; we do not use equality reflection to justify a restricted induction rule before reflection has been proved.

Both numerical interpretation domains are $x = x$. The interpreted equality is actual equality, and the numerical isomorphisms for the composites have graph $x = y$. Internal uniqueness of a fixed primitive recursion, proved by equation induction, identifies each double-translated operation with the original one. Structural induction on each fixed description then proves preservation of every operation; the native formula round trips give the logical composites as well. These claims concern arbitrary internal numbers, not merely standard numerals.

The precise downstream Lean statements in the pinned source include `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff` in `Lean/ConciseIPRA/NativeBiInterpretation.lean`. This package includes that source for reference. It does not claim that this file alone is a standalone Lean project, or that the new deterministic-introduction theorem has been compiled in Lean.

4 A failed shortcut and the boundary it exposes

The current automatic-opening proposals inspect a zero row and a first row. One tempting simplification is to keep only the first-row check. It is unsound. This matters when trying to combine the new introduction rule with those proposals.

Choose a primitive recursive f with

$$f(0) = f(1) = 0, \qquad f(2) = 1,$$

and let $B(x, y)$ be $x = f(y)$. In full iPRA,

$$D_{x,y}B = \exists x(x = f(0) \to \forall y\, x = f(y))$$

is provable with witness 1: its antecedent is false. The first row at a fresh parameter b is $b = 0$. An unchecked first-row probe would therefore replace the opened witness by 0. The resulting clause is

$$0 = 0 \to \forall y\, 0 = f(y),$$

which gives $0 = 1$ at $y = 2$.

Our literal naming matrix has a computation-provable zero row and is used with ordinary Open; it does not make this invalid inference. Nor do we silently compose it with automatic opening: opening a successful name directly at t can remove the fresh alias $b = t$ on which the present simulation relies. A composition needs an additional proof, rather than a simple replacement of one rule name by another.

5 Further research questions

1. **Can automatic introduction and automatic opening share a smaller bootstrap?** The present proof uses full formula transport and ordinary opening. A useful next target is a simultaneous recovery of transport, naming, and deterministic introduction in the three-family full-normal-form calculus. The dependence of existing equality proofs on unrestricted introduction must be removed explicitly.

2. **Can the deterministic selector be made cheaper to recognize?** Its mathematical data are smaller, but a pattern recognizer can exceed the source cost of validating a supplied term. A redesigned formula grammar may make the name structure intrinsic; any extra grammar and translation costs must be counted.

3. **Can full formula Tr be weakened before the bootstrap?** The algebraic-cut argument uses transport through an arbitrary guarded formula. Reducing it to term-only transport would require proving formula transport without already using unrestricted witnesses or unrepaired weakening.

4. **What is the proof-size cost?** The construction is effective, but this report does not claim a linear or polynomial bound on the serialized certificates. An explicit compiler could measure duplication caused by repaired weakening and naming.

5. **Can the new structural theorem be formalized against the existing Lean kernel?** A direct port using the substitution-stable diagonal presentation is plausible, but the guarded presentation and its exact source interpretation should first be pinned in the formal interface. A source-level prototype is not a compiled theorem.

6. **Which size metric captures the intended simplicity?** Rule-data count, complete typed-checker size, compact encoding size, and conceptual prerequisites reward different changes. A future minimum claim should fix one public input contract and one measured implementation language, and distinguish observed records from lower bounds.

6 Executable implementation and honest size comparison

6.1 The new calculus and its matched control

The selected mathematical verifier is `code/checker.py`. Its comparison program `code/control.py` has the same terms, formulas, clauses, computation axioms, Use, full formula Tr, Open, scope checks, JSON interface, and acceptance convention. The control takes an explicit term in Intro; the candidate instead calls the deterministic witness selector. The formulas are raw finite syntax: no normalization or external solver is hidden in the trusted boundary.

The complete certificate format is in `code/FORMAT.txt`. Numerical zero is the integer 0; a free parameter is a negative level; positive indices are de Bruijn bound variables. The numerical constructors are $[0, t]$ for successor and $[n, a, h]$ for iteration, binding only in h. An equation is $[s, t]$ and a paired formula is $[B]$, whose matrix binds two variables. The formula validator enforces the guard at every atomic paired matrix.

In this encoding a literal naming matrix is

$$[\,[1, 0, 3],\ [1, 0, r],\,],$$

provided r validates in the parent scope with no extra bound slots. The index 3 is the witness x beneath the iterator's state binder; the row and state occupy the nearer two slots. Internally bound indices of r remain legal. The scope check rejects references to the matrix's witness, row, or state even when those references are nested inside other iterators.

Whole standalone module	Bytes	Tokens	AST nodes
Unrestricted-witness control	3442	1078	1252
Deterministic introduction	3688	1159	1339
Difference	+246	+81	+87

These measurements count the entire acceptance program, including parsing, syntax, binding, scope, rule checking, and the final verdict. They use CPython 3.12.14, removal of actual docstrings, `ast.unparse`, and one final line feed. Significant tokens exclude whitespace, indentation, comments, and end markers; AST nodes are counted by `ast.walk`. The programs import only Python's standard library. Untrusted proof builders and audit scripts are outside both checker measurements.

The new mathematical calculus is not smaller under this matched checker-size metric. It saves the independent witness datum at each introduction; recognizing the fixed naming exception costs more source in this implementation. The comparison is included to prevent the interface theorem from being mistaken for an achieved software minimum.

6.2 What the executable checks establish

The primary suite constructs ten accepted deterministic certificates and eleven control certificates, including an original proof using a compound iterator as its witness. It supplies an explicit deterministic proof of that identical conclusion. It also embeds each of the ten deterministic proofs into the control by restoring its inferred witness fields.

The most relevant structural fixture exhibits all three stages: an introduction valid before scope extension, a rejected unchanged replay after a newer parameter is inserted, and the successful naming repair. Other positives use an equation induction hypothesis and a compound case base

assumption. Eighteen negative certificates attack guards, pattern scope, bound-variable capture, witness escape, packet arities, and equality premises. The suite performs 78 standalone executions across ordinary and optimized Python, with five direct binding regressions.

An independently written targeted audit adds three positive and three negative fixtures, twelve additional ordinary/optimized executions, and five direct checks. It exercises full formula transport beneath one and two guarded D binders with nested iterator binders in the substituted term. It also rejects an opened parameter hidden in the unused body of a zero iterator; raw syntax cannot erase that escape.

The deterministic-to-control elaborator is complete for accepted finite certificates in this format. The opposite direction is established by the general mathematical proof and illustrated by an explicit translated pair; this archive does not claim a complete implemented compiler in that direction. All executed results are included as JSON. Finite tests cannot establish the universal bi-interpretation theorem, and no new proof-assistant verification is asserted.

6.3 A separate smaller implementation of the inherited calculus

The incoming native control performs a public-formula grammar traversal and then a scope traversal. The selected refinement checks grammar during the first scope traversal, using its existing depth parity as the formula/term distinction. After that first traversal, a local phase flag is zero, and every subsequent proof operation is the inherited operation.

The general idea of sharing these traversals already occurs in the repository [8]; the present change applies it to the newer continuation-fused control. It is an implementation refinement of an existing calculus. It does not implement the deterministic introduction of this paper.

Native public-goal-validated module	Bytes	Tokens	AST nodes
Incoming control	780	360	409
Shared initial traversal	656	333	379
Difference	−124	−27	−30

The refined complete source is 15.9% shorter in normalized bytes. It retains the same public-goal grammar and accepts the same successful certificate traces, including the same unused instruction prefix. It is not comparable as a single strict typing contract to the new, readable guarded checker above: the inherited compact protocol has its own documented administrative syntax. Its native public-goal validator must also be distinguished from complete validation of every raw proof field.

Universal equivalence argument. The inherited traversal starts at depth $d = 0$ and enters a list of length r by adding $5 - r$ to the depth. Formula lists have length 1 or 2; numerical lists have length 1 or 3. A formula D preserves even parity, an equation changes even to odd, and both numerical constructors preserve odd parity. Thus $d \bmod 2$ already is the separate validator's sort flag. During the initial pass the admissible list lengths are precisely

$$r \in \{1, 2 + (d \bmod 2)\}.$$

Term integers must be exactly 0 or odd integers at least 3. The compact test `not a or a > 1 == a % 2` expresses exactly this on actual integers, and a type test excludes booleans and floats.

The first-pass rejection path is forced to fail even at formula depth, where the ordinary scope traversal would otherwise have a neutral scalar fallback. Induction on the finite goal tree proves

that this pass succeeds exactly when both old passes succeed, and then returns the unchanged goal. A valid formula is a nonempty list and cannot equal the initial scalar context sentinel. Therefore the initial invocation necessarily clears its local phase flag before reading a proof packet. All recursive invocations start with flag zero. Their traversal and continuation expressions are exactly the old ones. Induction on the consumed packet suffix proves equality of successful traces. The full argument and finite checks are in `implementation/`.

The separate 506-byte compact profile. The archive also records a two-byte change to the inherited 508-byte guarded compact module: replace $k - 1$ by $\sim(-k)$. Python integer complement gives $\sim(-k) = k - 1$ for every integer, so all scopes and accepted traces remain the same. It has 506 bytes, 267 tokens and 303 AST nodes, compared with 508, 267 and 302. This is arithmetic source compression only, and actually adds one AST node. It supplies no new mathematical simplification and no public-goal validator. We do not present this two-byte saving as a breakthrough or as the implementation of the new theory.

7 Reproduction and deliverable map

The archive contains the standalone report source and compiled PDF, the editable component sources, the new checker and matched control, accepted and rejected certificates, proof elaboration examples, reproducible measurements, exact inherited comparison sources, the attributed arithmetic bridge, and a checksum manifest. It does not contain nested copies of older delivery archives or an unbuilt proof-assistant project.

Run from the extracted directory:

```
python3 verify_manifest.py
python3 reproduce.py
python3 build_report.py
python3 code/checker.py < code/examples/accepted/arbitrary_witness_translated.json
```

The standalone `ipra_deterministic_introduction.tex` also builds directly with pdfLaTeX; it has no external input files or graphics. The modular builder regenerates it from the editable sources and the normalized checker listing. Compilation disables shell escape and requires standard TeX Live packages. The Python verifiers use only the standard library. Measurements are tied to the stated Python version. All theoretical verifier claims concern ideal finite executions with sufficient resources; a concrete process may exhaust memory or recursion limits on a valid large proof.

8 Complete checker for the new calculus

This is the complete measured acceptance module after removal of its module docstring and normalization by `ast.unparse`. It imports no proof-producing or verification helper.

```
import json
import sys

def need(condition):
    if not condition:
        raise ValueError('invalid certificate')

def term(a, scope, depth=0):
    if type(a) is int:
        return -scope <= a <= depth
    if type(a) is not list:
        return False
    if len(a) == 2:
        return type(a[0]) is int and a[0] == 0 and term(a[1], scope, depth)
    return len(a) == 3 and term(a[0], scope, depth) and term(a[1], scope, depth) and term(a[2], scope, depth
      + 1)

def formula(a, scope, depth=0):
    if type(a) is not list:
        return False
    if len(a) == 2:
        return all((term(b, scope, depth) for b in a))
    if len(a) != 1 or not formula(a[0], scope, depth + 2):
        return False
    b = a[0]
    return len(b) == 1 or all((type(t) is list and len(t) == 3 and (t[:2] == [1, 0]) for t in b))

def binding(a, index):
    return 2 if len(a) == 1 else int(len(a) == 3 and index == 2)

def shift(a, amount, depth=0):
    if type(a) is int:
        return a + amount if a > depth else a
    return [shift(b, amount, depth + binding(a, i)) for i, b in enumerate(a)]

def sub(a, value, index=1, depth=0):
    if type(a) is int:
        if a == index + depth:
            return shift(value, depth)
        return a - 1 if a > index + depth else a
    return [sub(b, value, index, depth + binding(a, i)) for i, b in enumerate(a)]

def witness(matrix, scope):
    if len(matrix) == 2 and matrix[0] == [1, 0, 3]:
        r = matrix[1]
        if type(r) is list and len(r) == 3 and (r[:2] == [1, 0]) and term(r[2], scope):
            return r[2]
    return -scope
H = [[0, 0, 0], 0]

def infer(proof, clauses=(), scope=0):
    need(type(proof) is list and len(proof) > 0)
    tag = proof[0]
    if tag == 'zero':
        need(len(proof) == 3)
        a, h = proof[1:]
        need(term(a, scope) and term(h, scope, 1))
        out = [[0, a, h], a]
    elif tag == 'succ':
        need(len(proof) == 4)
        n, a, h = proof[1:]
```

```
        need(term(n, scope) and term(a, scope) and term(h, scope, 1))
        out = [[[0, n], a, h], sub(h, [n, a, h])]
    elif tag == 'use':
        need(len(proof) == 4)
        j, t, p = proof[1:]
        need(type(j) is int and 0 <= j < len(clauses) and term(t, scope))
        a, body = clauses[j]
        need(infer(p, clauses, scope) == a)
        out = sub(body, t)
    elif tag == 'tr':
        need(len(proof) == 4)
        template, p, q = proof[1:]
        need(formula(template, scope, 1))
        equation = infer(p, clauses, scope)
        need(len(equation) == 2)
        s, t = equation
        need(infer(q, clauses, scope) == sub(template, s))
        out = sub(template, t)
    elif tag == 'open':
        need(len(proof) == 3)
        major = infer(proof[1], clauses, scope)
        need(len(major) == 1)
        body = sub(major[0], -scope - 1, 2)
        out = infer(proof[2], clauses + ((sub(body, 0), body),), scope + 1)
    elif tag == 'intro':
        need(len(proof) == 3)
        matrix, p = proof[1:]
        need(formula([matrix], scope))
        t = witness(matrix, scope)
        z = -scope - 1
        body = sub(matrix, t, 2)
        base = sub(body, z if len(matrix) == 2 else 0)
        need(infer(p, clauses + ((H, base),), scope + 1) == sub(body, [0, z]))
        out = [matrix]
    else:
        raise ValueError('unknown instruction')
    need(formula(out, scope))
    return out

def check(data):
    need(type(data) is dict and set(data) == {'goal', 'proof'})
    need(formula(data['goal'], 0))
    need(infer(data['proof']) == data['goal'])
if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
        print('ACCEPT')
    except Exception:
        print('REJECT')
        sys.exit(1)
```

References

[1] V. Reshetnikov, ed., *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt, pinned commit `11814771b7490dfbea0ed5fcf0f5b23586334764`. Repository snapshot.

[2] *A guarded core with a deterministic introduction rule*, `Research/guarded_core/sections/guarded_core.tex`, in [1]. This is the inherited guarded formula language, root-dependent mode, and guarded/native comparison; its witness term is still supplied.

[3] *Complementary introductions with one local assumption*, `Research/certified_zero/reference/inherited_choice_basis.tex`, in [1]. Explicit native computation, clause, equality, and Choice-DI rules.

[4] *Inherited ordinary bridge*, `Research/certified_zero/reference/inherited_ordinary_bridge.tex`, in [1]. Ordinary logical reconstruction and the scalar realization.

[5] *Scalar predecessor*, *Equality reflection*, and *Primitive recursion*, files `scalar_predecessor.tex`, `equality_reflection.tex`, `primitive_recursion.tex` under `Research/certified_zero/reference`, in [1]. The complete attributed source fragments are included in this archive.

[6] `Lean/ConciseIPRA/NativeBiInterpretation.lean` and its dependencies, in [1]. Exact formal interface. The new deterministic-witness theorem of this report is a written proof, not a new theorem in that pinned Lean source.

[7] The six archives under `docs/incoming`, in [1]: `ipra_clause_discharge_2026-11-11.zip`, `ipra_deterministic_opening_2026-11-11.zip`, `ipra_first_instance_2026-11-11.zip`, `ipra_inferred_witness_2026-11-11.zip`, `ipra_prefix_clauses.zip`, and `ipra_substitutive_discharge.zip`. The deterministic-opening archive contains the exact inherited 508-byte compact and 780-byte goal-validated controls used in this report. Its automatic-opening theorem is distinct from our automatic-introduction theorem.

[8] `Research/scope_kernel` and `Research/validated_core`, in [1]. Earlier work on sharing validation and scope traversal. The separate refined native checker here applies that implementation idea to the incoming continuation-fused control. It does not claim a new arithmetic or logical basis.